

TalkToData Assignment Plan

TalkToData Assignment Plan

Part 1: Comprehension Checkpoints

1. What is the real bottleneck at the client company?

The client does not have a software shortage; it has an access shortage. Business leaders know what they want to ask, but only a small data team can translate those questions into SQL, so routine decisions wait in a long queue instead of being answered when the business needs them.

2. Three ways to ask the same question, and why that breaks simple rules

Three managers could ask the same question in very different ways:

- How many orders came from Mumbai?
- How many purchases did we get from Mumbai?
- What was our Mumbai order count?

This variation makes a keyword-only system fragile because the intent stays the same while the wording changes. A brittle rule set might only recognize one phrasing, miss synonyms like "purchases," or fail when the sentence structure changes even though the business meaning has not.

3. What are the source language, target language, and grammar?

The source language is plain English and the target language is SQL, specifically SQLite SQL in this case study. The grammar the translation must obey is the database schema and relational structure: valid table names, valid column names, and the join relationships that connect the tables correctly.

4. Why is a wrong SQL query easier to catch than a wrong French sentence?

A wrong SQL query reveals itself in two practical ways. It either fails technically by throwing an execution error, or it runs successfully but produces numbers that are visibly inconsistent with the question, the schema, or business expectations.

5. What does trust require beyond returning a number?

Trust requires transparency, validation, and the ability to inspect what the system actually did. Before acting on a number, I would want to see the generated SQL, confirm that it used the correct tables and filters, know whether the query passed safety checks, and have enough context to decide whether the

answer is faithful to the original business question.

Part 2: Five-Question Mapping Table

	Tables needed	Join required?	Filters	Grouping / aggregate
	`orders`	No	`city`, date range on `order_date`	`COUNT(*)`
P	`order_items`, `products`	Yes	None	`SUM(quantity * price)` grouped
s?	`orders`, `customers`	Yes	`customers.is_repeat = 1`	`AVG(orders.total_amount)`
	`returns`, `orders`, `order_items`, `products`	Yes	None	`COUNT(returns.id)` grouped by
	`customers`	No	None	`COUNT(*)` grouped by `city`

Phase Predictions

Phase A: Rule-based NL-to-SQL

I expect Phase A to handle only the simplest patterns reliably, especially direct counting questions like the Mumbai orders example or a basic customer count. I expect it to break when phrasing changes, when time windows like "last week" matter, and whenever a question needs joins, grouping, or ranking.

Phase B: LLM-based NL-to-SQL

I expect Phase B to perform much better on complex questions because it can infer joins, aggregates, and grouping without me hard-coding every rule. I also expect it to fail in a more dangerous way: it may generate fluent, valid SQL that ignores part of the question, invents a table or column, or answers a slightly different business question while still looking correct.

Phase C: LLM + Guardrails

I expect Phase C to be the strongest overall because it preserves the flexibility of the LLM while adding safety checks and transparency. It still will not make the system perfect, but it should make failure modes much more visible and much less risky in practice.

Production Recommendation

The phase I would trust most in production is Phase C, because the client needs both translation capability and verifiable safety: the model can generate useful SQL, while guardrails and mandatory SQL review make wrong-but-valid outputs easier for a human to catch before a business decision is made.